DBMS

PL-SQL Lab 1

Pavithra Rajeev | AM.SC.U4AIE23144

1. Do all the questions discussed in the class

1. Write a function to add 2 integers values and return sum

```
--Q1--
create or replace function sumint(a int, b int)
    returns int as
$$
    Declare
        c int;
    Begin
        c =a+b;
        return c;
    End;
$$
Language 'plpgsql';

select sumint(44, 2)
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	sumint integer
1	46

2. Write a function that accepts an integer value and returns odd or even.

```
--Q2--
create or replace function check_even_odd(a int)
    returns varchar as
$$
    Begin
        if a % 2 =0 then
            return 'even';
        else
            return 'odd';
        End if;
    End;
$$
Language 'plpgsql';

select check_even_odd(44)
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	check_even_odd character varying
1	even

Create tables Dept and Emp.

	dept_no [PK] integer	dname character varying (50)
1	10	CS
2	20	EEE
3	30	MEE
4	40	AI

	eno [PK] integer	ename character varying (50)	sal numeric (10,2)	dno integer
1	1	Pavithra	50000.00	10
2	2	Swetha	55000.00	20
3	3	Resh	54000.00	30
4	4	Kashi	60000.00	40
5	5	Deva	60000.00	10
6	6	Keerthana	40000.00	20
7	7	Nira	40000.00	10

1.Write a function that accepts an employee number and returns the salary of that employee.

```
--1--
create or replace function emp_sal(emp_no Emp.eno%type)
    returns Emp.sal%type as
$$
    Declare
        salary Emp.sal%type;
    Begin
        select sal into salary from Emp where eno=emp_no;
        return salary;
    End;
$$
Language 'plpgsql';

select emp_sal(2)
```

Data Output   Messages   Notifications

	emp_sal numeric
1	55000.00

2.Write a function that accepts a depno and finds the no. of employees in that department.

```sql
--2--
create or replace function no_of_employees(mydep_no Emp.dno%type)
    returns int as
$$
    Declare
        countemp int;
    Begin
        select count(eno) into countemp from Emp where mydep_no=dno;
        return countemp;
    End;
$$
Language 'plpgsql';

select no_of_employees(10);
```

Data Output Messages Notifications

	no_of_employees integer
1	3

3. Write a function that accepts an employee number and give that employee 10% high in salary if average salary of the corresponding department is greater than employees current salary.

```sql
--3--
create or replace function increase_sal(emp_no Emp.eno%type)
    returns varchar as
$$
    Declare
        salary Emp.sal%type;
    Begin
        select sal into salary from Emp where eno=emp_no;
        if (salary< (select avg(sal) from Emp where dno= (select dno from Emp where eno=emp_no))) then
            update Emp set sal = sal + (sal*(0.10)) where eno=emp_no;
        End if;
        return 'salary updated';
    End;
$$
Language 'plpgsql';
select increase_sal(3)
```

Data Output Messages Notifications

	increase_sal character varying
1	salary updated

4. Write a function that accepts an employee name and returns the dep name of the employee (assuming all names are unique)

```sql
--4--
create or replace function dep_name(empname Emp.ename%type)
    returns Dept.dname%type as
$$
    Declare
        mydeptname Dept.dname%type;
    Begin
        select D.dname into mydeptname from Emp E join Dept D on E.dno = D.dept_no where E.ename= empname
        return mydeptname;
    End;
$$
Language 'plpgsql';

select dep_name('Pavithra');
```

Data Output Messages Notifications

	dep_name character varying
1	CS

2. Consider the following schema .

Emp(eno, ename, sal)

EMP_PROJ(eno, pno, prjt_hrs).

	eno [PK] integer	ename character varying (50)	sal numeric (10,2)	dno integer
1	1	Pavithra	50000.00	10
2	2	Swetha	55000.00	20
3	3	Resh	54000.00	30
4	4	Kashi	60000.00	40
5	5	Deva	60000.00	10
6	6	Keerthana	40000.00	20
7	7	Nira	40000.00	10

	eno integer	pno integer	prjt_hrs numeric (10,2)
1	1	101	12.50
2	1	102	15.00
3	2	103	20.00
4	3	104	18.00
5	4	105	10.00
6	5	106	22.00
7	6	107	16.50
8	7	108	14.00
9	7	109	19.00

a) Write a function ProjectLoad that returns the total project working hours for the given eno.

```
--a--
create or replace function projectload(emp_no emp.eno%type)
    returns numeric as
$$
    Declare
        total_hrs numeric;
    Begin
        select coalesce(sum(prjt_hrs)) into total_hrs from emp_proj where eno = emp_no;
        return total_hrs;
    end;
$$
language 'plpgsql';

select projectload(1);
```

Data Output Messages Notifications

	projectload numeric
1	27.50

b) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary. New salary is current sal + difference between currrent sal and average salary.

```
--b--
create or replace function update_sal(emp_no emp.eno%type)
    returns varchar as
$$
    Declare
        avg_sal numeric;
        curr_sal numeric;
        new_sal numeric;
    Begin
        select sal into curr_sal from emp where eno = emp_no;
        select avg(sal) into avg_sal from emp;
        if curr_sal < avg_sal then
            update emp
            set sal = sal + (avg_sal - curr_sal)
            where eno = emp_no;
        end if;
        select sal into new_sal from emp where eno = emp_no;

        return new_sal;
    end;
$$
language 'plpgsql';

select update_sal(3);
```

Data Output Messages Notifications

	update_sal character varying
1	54000.00

3. Consider the following tables Item(ino, iname, unit_price) Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

	ino [PK] integer	iname character varying (50)	unit_price numeric (10,2)
1	101	Puffs	10.00
2	102	Burger	50.00
3	103	Pizza	5.00
4	104	Roll	8.00
5	105	Custard	25.00

	tr_no [PK] integer	ino integer	qty integer
1	1	101	5
2	2	101	3
3	3	102	10
4	4	103	2
5	5	103	4
6	6	104	1

```sql
create or replace function delete_item(ino_input item.ino%type)
    returns varchar as
$$
    Declare
        trans_count int;
    Begin
        select count(*) into trans_count from transaction where ino = ino_input;
        if trans_count < 2 then
            delete from item where ino = ino_input;
            return 'item deleted';
        else
            return 'item not deleted';
        end if;
    end;
$$
language 'plpgsql';

select delete_item(101);
```

Data Output Messages Notifications

	delete_item character varying
1	item not deleted

4. Consider a Bank database which includes the following tables. ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal) BRANCHES(br_no, br_name, loc) CUSTOMER(cno, cname, c_type)

	cno [PK] character varying (10)	cname character varying (50)	c_type character varying (10)
1	C1	PAVITHRA	A
2	C2	SWETHA	B
3	C3	RESH	A
4	C4	KASHI	B

	br_no [PK] character varying (10)	br_name character varying (50)	loc character varying (50)
1	BR1	MAIN BRANCH	CHENNAI
2	BR2	CITY BRANCH	BANGALORE
3	BR3	TOWN BRANCH	MUMBAI

	ac_no [PK] character varying (10)	br_no character varying (10)	cust_no character varying (10)	ac_type character varying (20)	bal numeric (12,2)
1	A1	BR1	C1	SAVINGS	60000.00
2	A2	BR1	C2	CURRENT	30000.00
3	A3	BR2	C3	SAVINGS	75000.00
4	A4	BR2	C1	CURRENT	20000.00
5	A5	BR3	C4	SAVINGS	15000.00
6	A6	BR3	C2	CURRENT	50000.00

a) Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.

```sql
--a--
create or replace function update_custype(threshold numeric, cust_id customer.cno%type)
    returns varchar as
$$
    Declare
        total_bal numeric;
    Begin
        select sum(bal) into total_bal from accounts where cust_no = cust_id;

        if total_bal > threshold then
            update customer set c_type = 'A' where cno = cust_id;
        else
            update customer set c_type = 'B' where cno = cust_id;
        end if;
        return 'customer type updated';
    end;
$$
language 'plpgsql';
select update_custype(50000.00, 'C1')
```

Data Output Messages Notifications

	update_custype character varying
1	customer type updat...

b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

```sql
--b--
create or replace function closebranch(close_br branches.br_no%type, new_br branches.br_no%type)
    returns varchar as
$$
    Declare
        cnt int;
    Begin
        update accounts set br_no = new_br where br_no = close_br;
        delete from branches where br_no = close_br;

        return 'branch closed and accounts transferred';
    end;
$$
language plpgsql;
select closebranch('BR1', 'BR3')
```

Data Output Messages Notifications

	closebranch character varying
1	branch closed and accounts transferr...

c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```sql
--c--
create or replace function safe_withdraw(accno accounts.ac_no%type, amt numeric)
    returns varchar as
$$
    Declare
        curr_bal numeric;
    Begin
        select bal into curr_bal from accounts where ac_no = accno;

        if curr_bal >= amt then
            update accounts set bal = bal - amt where ac_no = accno;
            return 'withdrawal successful';
        else
            return 'insufficient funds - withdrawal denied';
        end if;
    end;
$$
language plpgsql;
select safe_withdraw('A1', 30000)
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	safe_withdraw character varying
1	withdrawal successf...